

第一节正式课

1. `input()` #可以让用户输入信息

例如: `input("请输入你的名字")`

```
Box = input("请输入你的名字")
```

引号里输入提示信息, 可以把输入的名字放到一个盒子中

2. `print()` #展示、显示结果, 就像打印机一样

例如: `print("垃圾分类")` #运行展示结果为"垃圾分类"

```
print(Box) #展示的结果为盒子里放的内容
```

3. 注释就是对代码的解释说明, 使用方法是在代码或中文说明的前面加上"#", 多行注释使用"Ctrl + #".

4. 代码编程实在英文状态下编辑字母和字符的。

第二节正式课

1. `import turtle` #导入、找到海龟

2. `t = turtle.Pen()` #找到海龟画笔

3. `t.shape("turtle")` #显示海龟形状, `shape` 形状

4. `t.forward()` #让海龟前进一段距离

5. `t.left()` #让海龟左转一个角度

6. `t.right()` #让海龟右转一个角度

7. `t.pencolor("gold")` #改变画笔颜色

8. `t.circle(radius, steps)` #以给定半径画圆

`radius`(半径): 半径为正(负), 表示圆心在画笔的左边(右边)画圆。

`steps`(做半径为 `radius` 的圆的内切正多边形, 多边形数为 `steps`)

例如: `t.circle(100)` #画一个圆心在画笔左边、半径为100的圆。

`t.circle(100, steps=6)` #画一个半径为100的圆的内切正六边形。

第三节正式课

1. `t.fillcolor("gold")` #确认需要填充的颜色
2. `t.begin_fill()` #开始填充
3. `t.end_fill()` #结束填充
4. `t.up()` #把海龟的画笔抬起来
5. `t.goto()` #将海龟移动到为个位置
6. `t.down()` #将海龟的画笔放下
7. `t.pensize()` #设置画笔的线宽
8. `t.bgcolor()` #设置背景颜色
9. `t.hideturtle()` #隐藏画笔
10. `t.speed()` #设置运行速度

取值区间为 0-10, 0 的速度最快, 1-10 速度依次递增

第四节正式课

1. **t.setheading(angle)** #设置画笔初始角度

angle 角度, 角度为正, 逆时针旋转, 角度为负, 顺时针旋转。

步骤分为两步:

① 先将小海龟头部方向改变为水平向右

② 然后根据角度正负进行逆时针、顺时针旋转。

2. **t.circle(100, 180)**

#画一个半径为 100, 角度为 180° 的圆弧, 即半圆。

#半径和圆弧度数前可加负号调节半圆位置, 注意配合转向使用。

3. **t.write()** #在画布上写文本

例如:

t.write(" 垃圾分类 ", align="center", font=(" 楷体", 50, "bold"))

align 设置位置, center 中心, left 左边, right 右边

font 字体, "楷体"设置字体类型, 如: 宋体、黑体、华文琥珀、华文彩云等。

50 则为字体的大小

"bold" 设置字体是否加粗, 可写可不写

注意括号是有两个嵌套在一起的

第五节正式课

1. **字符串**: 双引号或单引号之间的任意文本。

2. 字符串的组成：数字、文字、字母、符号。
3. **变量**：编程中最基本的存储单位，临时存放数据。
4. "+"两边是字符串或存放了字符串的变量时，表示连接符。

例如：`print("2019"+"年")` #显示：2019 年

`print("2019", "年")` #显示：2019 年

`a = "2019"`

`b = "年"`

`print(a+b)` #显示：2019 年

5. "="在编程中表示赋值号，可用来给变量赋值。
6. 变量命名规则：
 - ①变量名区分大小写
 - ②变量名必须以字母或下划线开头，不能以数字开头
 - ③变量名中不能包含空格
7. 变量是可以改变的，后放入的数据会覆盖放入前面的数据。

第六节正式课

1. 算术运算符：
 - ① `+`、`-`、`*`、`/`，除法运算结果是一个"浮点数"。
 - ② `//`，取整数，返回相除后结果的整数部分。
 - ③ `%`，取模，又叫求余，返回除法的余数。
 - ④ `**`，幂运算，返回 x 的 y 次幂。y 个 x 相乘。

例如：`print(2**3)` # 2 的 3 次幂，结果为 8

数学上写法为 2^3 。

电脑键盘上方法是： 2^3 。

2. 赋值运算符：

① $+=$ ，自增，例如：`a = 3 a+=1 print(a)` #结果为 4。

② $-=$ ，自减，例如：`a = 3 a-=1 print(a)` #结果为 2。

类似：`a = 3 a = a + 1 print(a)` #结果为 4

相应还有 $*=$ 和 $/=$ 。

3. 比较运算符：

① $==$ ，等于，比较两个对象是否相等，相等为 **True**，不相等为 **False**。

② $!=$ ，不等于，比较两个对象是否不相等，相等为 **False**，不相等为 **True**。

例如：`a = "15"`

`b = "12"`

`print(a == b)` #结果为 **False**

`print(a != b)` #结果为 **True**

4. 运算符的优先级：

运算符	运算优先级	注意
**		有括号先算 括号里的
*、/、%		
+、-		

从高到低

第七节正式课

1. 数据的类型:

① **整数**: 包含正整数、零、负整数。例如: **-1、0、1**。

② **浮点数**: 指数学中的小数。例如: **-1.5、0.8、1.50**。

③ **字符串**: 双引号或单引号之间的任意文本。例如:
"kfc"、"123"、"编程"。

2. **二进制**数据是用 **0** 和 **1** 两个数码来表示的数。

它的基数为 **2**，进位规则是“**逢二进一**”，借位规则是“**借一当二**”。

参考生活中常用“**十进制**”，进位规则是“**逢十进一**”，借位规则是“**借一当十**”。

例如:二进制数“**0001**”代表十进制的“**1**”

二进制数“**0010**”代表十进制的“**2**”

二进制数“**0011**”代表十进制的“**3**”

以此类推。

3. 数据的转换:

① **int()**，将一个浮点数或字符串转换为整数类型，也叫整型。注意: **int()** 将一个浮点数转化为整数时，正数**上取整**，负数**下取整**。

② **float()**，将一个整数或字符串转换为浮点数类型，也叫浮点型。

③ **str()**，将一个数据(任何类型)转换为字符串类型。

4. `print(type())` # 查看数据类型

5. 数据转换案例(仅供参考):

```
a = 6
b = 1.5
c = "18"          # 注意: 引号里必须是数字才可以进行转换。

# 查看转换前的数据类型
print(type(a))    # 结果为: <class 'int'>
print(type(b))    # 结果为: <class 'float'>
print(type(c))    # 结果为: <class 'str'>

# 将一个浮点数或字符串转换为整数类型
# 可辅助使用"type()"来查看转换后的数据类型
b1 = int(b)
c1 = int(c)
print(b1)         # 结果为: 1
print(type(b1))   # 结果为: <class 'int'>
print(c1)         # 结果为: 18
print(type(c1))   # 结果为: <class 'int'>

# 将一个整数或字符串转换为浮点数类型
# 可辅助使用"type()"来查看转换后的数据类型
a1 = float(a)
c2 = float(c)
print(a1)         # 结果为: 6.0
print(type(a1))   # 结果为: <class 'float'>
print(c2)         # 结果为: 18.0
print(type(c2))   # 结果为: <class 'float'>

# 将一个整数或浮点数转换为字符串类型
# 可辅助使用"type()"来查看转换后的数据类型
a2 = str(a)
b2 = str(b)
print(a2)         # 显示为: 6, 实际为: "6"
print(type(a2))   # 结果为: <class 'str'>
print(b2)         # 显示为: 1.5, 实际为: "1.5"
print(type(b2))   # 结果为: <class 'str'>
```

6. 结合 `input()` 使用:

注意: 使用 `input()` 输入的任何数据, 初始均为字符串类

型。

① `a = int(input("请输入一个整数"))` # 转化为整数

② `a = float(input("请输入一个小数"))` # 转化为浮点数

③ `a = int(input("请输入一个小数"))` # 这种是错误的，`int()` 无法将一个 **小数型的字符串** 转换为整数。

第八节正式课

1. **条件语句** 是一种根据条件执行不同代码的语句，**如果** 条件满足则执行一段代码，**否则** 执行其他代码。可将条件语句认为是有点像起因和结果。

生活举例：“如果你的房间是干净的，你会得到甜点。否则，你就得早点去睡觉。”

第一个起因是干净的房间，结果是可以得到甜点。第二个起因是不干净的房间，结果是必须早点上床休息。

2. **布尔值**：通常是表示**判断语句**的结果，有两种状态，结果为 **True** 表示**真**，结果为 **False** 表示**假**，注意首字母大写。

例如：

① 登录网站，经判断密码正确(**True**)，登陆成功。

② 登录网站，经判断密码错误(**False**)，登陆失败。

3. 比较运算符：

① **==**，左右两边等值的时候返回 **True**，否则 **False**。

② **!=**，左右两边不相等的时候返回 **True**，否则 **False**。

③ $>$ ，左边大于右边的时候返回 **True**，否则 **False**。

④ $<$ ，左边小于右边的时候返回 **True**，否则 **False**。

⑤ $<=$ ，左边小于或等于右边的时候返回 **True**，否则 **False**。

⑥ $>=$ ，左边大于或等于右边的时候返回 **True**，否则 **False**。

4. **if** 语句：做条件判断，测试条件真假。注意缩进。

例如： **age = 10**

```
if age < 18:      # 年龄小于 18 岁
    print("未成年人禁止入内")
```

5. **if-else** 语句：如果条件为真，执行 **if** 语句下的指令，否则(条件为假)就执行 **else** 语句下的指令。

例如： **age = 10**

```
if age >= 18:    # 年龄大于或等于 18 岁
    print("欢迎来到恐怖屋！")
else:           # 年龄小于 18 岁
    print("未成年人禁止入内")
```

第九节正式课

1. **if-elif-else** 语句在测试多个条件时使用。

例如： **age = float(input("请输入你的年龄："))**

```
if age < 4:      # 年龄小于 4 岁
    print("婴幼儿")
```

```
elif 4 <= age < 18:    # 年龄在 4-18 岁之间
    print("未成年")

else:                  # 年龄大于或等于 18 岁
    print("成年人")
```

2. 逻辑运算符:

① **and(与)**: 两边的条件, 只要有一个是 **False(假)**, 结果就是 **False(假)**。必须两边的条件都是 **True(真)**, 结果才是 **True(真)**。

② **or(或)**: 两边的条件, 只要有一个是 **True(真)**, 结果就是 **True(真)**。必须两边的条件都是 **False(假)**, 结果才是 **False(假)**。

③ **not(非)**: **not** 放在条件前面。如果条件是 **True(真)**, 最后结果就是 **False(假)**。如果条件是 **False(假)**, 最后结果就是 **True(真)**。**not(非)** 可以反转布尔值结果。

3. 图表示例:

A	B	A and B	A or B	notA
True	True	True	True	False
True	False	False	True	
False	True	False	True	True
False	False	False	False	

第十节正式课

1. **循环**: 通过**循环语句**让计算机执行某个重复的任务。
2. **循环语句**: 包括 **for-in** 循环和 **while** 循环。
3. **for-in 循环**: 固定次数的循环方式, 也叫**计数循环**。

4. for-in 循环结构:

for 循环变量 in range(循环次数):

```
|          +-----+  
| 缩进四格 | 要重复执 |  
|          | 行的步骤 |  
|          +-----+
```

5. 循环变量: 和其他变量相同, 只是名字不一样, 也称循环计数器, 通常使用 i、j、k 等作为循环变量。

例如:

```
for i in range(5):  
    print("聊天开挂模式启动!")
```

结果为连续 5 行的"聊天开挂模式启动!"。

6. range(): 用来生成一系列的数字, 通常用在循环中, 每循环一次, 从 range() 中得到一个数字放入循环变量中。

① range(x): 循环变量 i 从 0 开始, 到 x 前一个数字结束。

range(x) 中包括的数字为: 0....., x-1

例如: range(5) 中包括的数字为: 0、1、2、3、4

② range(x, y): 循环变量 i 从 x 开始, 到 y 前一个数字结束。

range(x, y) 中包括的数字为: x....., y-1

例如: range(1, 5) 中包括的数字为: 1、2、3、4

③ range(x, y, z): 循环变量 i 从 x 开始, 到 y 前一个数字结束, 每个数之间间隔为 z。z 称之为步长, 步长为 1, 默认不

写。

`range(x, y, z)` 中 包 括 的 数 字 为 :

`x, x+z, x+z+z, x+z+z+z....., y-1`

例如: `range(1, 5, 2)` 中包括的数字为: 1、3

`range(1, 8, 2)` 中包括的数字为: 1、3、5、7

`range(1, 11, 3)` 中包括的数字为: 1、4、7、10

7. 如下示例:

代码	描述
<code>for i in range(5):</code>	循环5次, 循环变量i 从0到4
<code>for i in range(0):</code>	循环0次
<code>for i in range(0,5):</code>	循环变量i 从0到4
<code>for i in range(1,5):</code>	循环变量i 从1到4
<code>for i in range(-2,3):</code>	循环变量i 从-2到2
<code>for i in range(-2,3,2):</code>	循环变量i 从-2到2, 每次循环i+2
<code>for i in range(0,-3,-1):</code>	循环变量i 从0到-2, 每次循环i-1

第十一节正式课

1. **while 循环**: 不限制循环次数, 用一个条件来确定是否结束循环, 也叫**条件循环**。

2. **while 循环结构**: **判断条件**为 **True**, 循环一直执行, **判断条件**为 **False**, 循环结束。

While 判断条件:

```
|          +-----+
| 缩进四格 | 要重复执 |
|          | 行的步骤 |
```

| +-----+

3. 参考代码：

```
age = int(input("请输入你的年龄："))
while age<18:
    a = 18-age
    print("你现在的年龄是：" +str(age)+"还需要"+str(a)+"年才可以考驾照哦！")
    age+=1
    print("一年后.....")
print("恭喜你，可以去考驾照了！")
```

第十二节正式课

1. 当 **while** 循环开始后，先判断条件是否满足，如果**条件满足(True)**就执行循环体内的语句，执行完毕后再回来判断条件是否满足，如此**无限重复**；直到**条件不满足(False)**时，执行 **while** 循环后边的语句。

2. 无限循环结构：

While True:

| +-----+
| **缩进四格** | 要重复执 |
| | 行的步骤 |
| +-----+

3. 无限循环示例：

```
b = 0
while True:
    a = input("请输入你的名字：")
    b+=1
    print("有"+str(b)+"个人通过路口")
```

4. **死循环**：指在编程中，一个靠自身控制无法终止的程序。

死循环是不符合我们目的的**无限循环**。

5. **break 跳出循环**：在必要的时候让程序结束。

6. **end=""**表示末尾不换行，添加一个空字符串，表示这个语句还未结束。

例如：**print("人工", end="")**

print("智能")

显示结果为：**人工智能**

而不是：**人工**

智能

7. **\n**：表示换行，相当于回车。

例如：**print("人工\n智能")**

显示结果为：**人工**

智能

而不是：**人工智能**

8. **%s**：占位符，插入一个字符串变量。

例如：**a = 12**

b = 13

print("a 是%s"%a) # 显示结果为：**a 是 12**

print("a 是%s, b 是%s"%(a, b))

显示结果为：**a 是 12, b 是 13**

9. **continue 语句**：跳过某次循环，下面语句不再执行，并继

续执行下一次循环。

例如：

```
a = 0
while True:
    a+=1
    if a > 10:
        break
    elif a == 5:
        continue
    print(a)
```

如果 a 的值是 5，跳过本次循环，继续执行下一次循环
导致的结果即是下方的 print(a) 无法执行，结果部分
不会打印出本次循环的 a 的值 5。

第十三节正式课

综合运用，熟悉前面的课程知识。

第十四节正式课

1. **列表**：使用**方括号**表示，其内可以放入**任何数据类型**，用**"="**创建列表，给变量赋值，**列表**是可变的。
2. **列表**由一系列按**特定顺序**排列的**元素**组成，列表中的单个元素叫**项**。使用**逗号**分隔列表的**各项**。

例如：

创建列表

```
color_list = ["red", "blue", "green", "gold"]
```

展示列表

```
print(color_list)
```

3. **索引**：每一个项在列表中都有一个对应的索引值，代表着列表中元素的位置，索引有正负之分，**正索引**从左向右，**负索引**从右向左。

列表项	"编小美"	"哈乐"	"杭小宝"	"妙小程"
正索引值	0	1	2	3
负索引值	-4	-3	-2	-1

4. 列表查询：列表名+[索引值]

例如：`color_list = ["red", "blue", "green", "gold"]`

```
print(color_list[1])    # 显示为 blue
```

```
print(color_list[-1])   # 显示为 gold
```

```
print(color_list[1:3])
```

```
# 显示为 ["blue", "green"]
```

5. 列表添加：

① `append(object)` 在列表对象的末尾加上新的对象。

例如：`color_list = ["red", "gold"]`

```
color_list.append("orange")
```

```
print(color_list)
```

```
# 显示为 ["red", "gold", "orange"]
```

② `extend(list)` 将 `list` 列表中的元素加入另一个列表中。

例如：`color_list = ["red", "gold"]`

```
A = ["black", ["white"]]
```

```
color_list.extend(A)
```

```
print(color_list)
```

```
# 显示为 ["red", "gold", "black", ["white"]]
```

③ `insert(index, object)` 在列表中索引值为 `index` 的元

素前插入新元素 **object**。

例如: `color_list = ["red", "gold"]`
`color_list.insert(1, "blue")`
`print(color_list)`
显示为 `["red", "blue", "gold"]`

6. 列表删除:

① `remove(value)` 将列表中的元素值 **value** 删除

例如: `color_list = ["red", "gold", "blue"]`
`color_list.remove("gold")`
`print(color_list)`
显示为 `["red", "blue"]`

② `pop(index)` 将列表中索引值为 **index** 的元素值删除,
如果没有指定的索引值, 就默认删除**最后一个**元素。

例如: `color_list = ["red", "gold", "blue", "green"]`
`color_list.pop(1)`
`print(color_list)`
显示为 `["red", "blue", "green"]`
`color_list.pop()`
`print(color_list)`
显示为 `["red", "blue"]`

7. 列表修改: 利用通过索引值可以查找到**元素值**的方式进行
重新赋值。

例如: `color_list = ["red", "gold", "blue", "green"]`

```
color_list[2] = "gray"
```

```
print(color_list)
```

```
# 显示为 ["red", "gold", "gray", "green"]
```

8. **遍历列表**: 用 **for-in** 循环依次得到列表、元组、和字符串中的元素这个过程称之为**遍历**。

例如: `color_list = ["red", "gold", "blue", "green"]`

```
for i in color_list:
```

```
    print(i)
```

```
# 显示结果为: red
```

```
    gold
```

```
    blue
```

```
    green
```

第十五节正式课

1. **嵌套列表查询**: 每深入一层, 就多一个索引值。

例如:

```
A = [1, 2, 3, ["q", "w", "e"]
```

```
print(A[3][1])    # 结果为 w
```

2. **列表排序**:

① **sort()**, 将列表从大到小排序, 改变原列表

例如:

```
A = [2, 1, 23, 12, 5, 19]
```

```
A.sort()
```

```
print(A)    # 显示结果为 [1, 2, 5, 12, 19, 23]
```

② **sorted()**, 将列表从大到小排序, 不改变原列表

例如:

```
A = [2, 1, 23, 12, 5, 19]
```

```
B = sorted(A)
```

```
print(A)    # 显示结果为 [2, 1, 23, 12, 5, 19]
```

```
print(B)    # 显示结果为 [1, 2, 5, 12, 19, 23]
```

③ **reverse()**, 将列表中的元素颠倒排列

例如:

```
A = [2, 1, 23, 12, 5, 19]
```

```
A.reverse()
```

```
print(A)    # 显示结果为 [19, 5, 12, 23, 1, 2]
```

3. 复制列表: 将旧列表中的**所有元素**赋值给新列表

例如:

```
A = [2, 1, 23, 12, 5, 19]
```

```
B = A[:]
```

```
print(B)    # 显示结果为 [2, 1, 23, 12, 5, 19]
```

4. **元组**: 与列表类似, 不同之处在于元组**不能改变**。元组使用**小括号**表示, 使用**逗号**隔开每一个元素, 注意: 当元组中只有一个元素的时候, 元素后面必须加上逗号。

5. 元组的查询：和列表一样，都是根据索引值来查询。

例如：

```
A = (2, 1, 23, 12, 5, 19)
print(A[2]) # 显示结果为 23
```

6. `in`、`not in`：成员运算符，检查某个元素是否在(不在)元组、列表、或字符串中。

例如：

```
A = "delete"
print("l" in A) # 显示结果为 True

B = [1, 2, 3, 4, 5]
print(2 in B) # 显示结果为 True

C = ("1", "2", "3", "4")
if "2" in C:
    print("T")
else:
    print("F")
# 显示结果为 T
```

7. 组合元组：元组中的元素无法修改，但是可以通过连接符进行连接组合。

例如：

```
A = ("1", "2", "3", "4")
B = (1, 2, 3, 5)
```

```
C = A+B
```

```
print(C)
```

```
# 显示结果为 ("1", "2", "3", "4", 1, 2, 3, 5)
```

第十六节正式课

1. **字典**：和列表、元组一样，一种用来**存储数据**的容器，字典用**键值对**的形式存储数据。
2. 字典将**键值(key)**和**数值(value)**联系在一起，中间用**:**隔开，使用**花括号{}**列出所有元素，用**逗号**分隔不同的项。

例如：

```
A = {"小红": "100", "小明": "80"}
```

3. **字典查询**：

例如：

```
A = {"小红": "100", "小明": "80"}
```

```
print(A["小红"]) # 显示结果为 100
```

4. **字典删除**：

例如：

```
A = {"小红": "100", "小明": "80"}
```

```
del A["小明"]
```

```
print(A) # 显示结果为 {"小红": "100"}
```

5. **字典添加**：

例如：

```
A = {"小红": "100", "小明": "80"}  
A["小美"] = "90"  
print(A)  
# 显示结果为  
# {"小红": "100", "小明": "80", "小美": "90"}
```

6. 字典修改:

例如:

```
A = {"小红": "100", "小明": "80"}  
A["小红"] = "90"  
print(A)  
# 显示结果为 {"小红": "90", "小明": "80"}
```

7. 字典修改与添加的区别:

当字典里已经有我们要添加的键值时, 不会再添加重复的键值, 而是会修改键值对应的数值

当字典里没有我们要添加的键值时, 会在字典里添加一个新的键值对

8. 字典遍历:

① `keys()`, 使用字典中的所有键值创建一个列表对象

例如:

```
A = {"小红": "100", "小明": "80"}  
print(A.keys()) # 显示为 ["小红", "小明"]  
for key in A.keys(): # 遍历所有键值
```

```
print(key)
```

```
# 显示为:    小红  
            小明
```

② `values()`, 使用字典中的所有数值创建一个列表对象
例如:

```
A = {"小红": "100", "小明": "80"}  
print(A.values()) # 显示为 ["100", "80"]  
for value in A.values(): # 遍历所有键值  
    print(value)  
# 显示为:    100  
            80
```

③ `items()`, 使用字典创建一个以 (key, value) 为一组的
元组对象。

例如:

```
A = {"小红": "100", "小明": "80"}  
print(A.items())  
# 显示为 [("小红", "100"), ("小明", "80")]  
for key, value in A.items(): # 遍历所有键值  
    print("姓名: %s"%key)  
    print("成绩: %s"%value)  
# 显示为:    姓名: 小红  
            成绩: 100
```

姓名：小明

成绩：80